

# Multi-Objective Optimization for Takt Planning with Spatial Zoning and Crew Resource Flow

1<sup>st</sup> Ashmitha Jaysi Sivakumar  
MS Sustainable Design and Construction  
Stanford University

**Abstract**—Construction scheduling is a highly combinatorial problem in which zone definitions, trade sequencing, crew allocation, and BIM-derived quantity constraints interact to affect duration, idle time, spatial feasibility, and resource use. This project develops a BIM-informed multi-objective optimization framework that generates takt zones, assigns crew resources, and evaluates schedules using NSGA-II with diagnostic crossover and mutation operators. Compared with a naive two-room zig-zag baseline, which produced a schedule duration of 258.82 hours, 464.92 crew-hours of idle time, and a connectedness penalty of 2.00, the best optimized variant reduced duration to 162.80 hours and idle time to 88.53 crew-hours while eliminating the connectedness penalty. This demonstrates that the proposed optimizer can substantially improve workflow reliability and schedule performance, while explicitly exposing the tradeoff with crew and resource usage.

**Index Terms**—Takt planning, Construction scheduling, Multi-objective optimization, BIM, NSGA-II

## I. INTRODUCTION

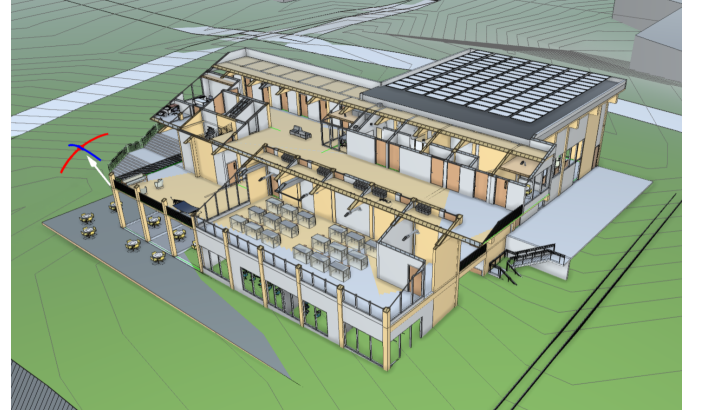
A construction project can be viewed as a coordination problem in which multiple trades, such as framing, mechanical, electrical, plumbing, drywall, and finishes, must move through the same physical building in a planned sequence. A construction schedule defines what work happens, where it happens, when it happens, and which crews or equipment are assigned to it.

Designing this schedule is highly combinatorial. Planners must decide how to divide the building into work zones, how to order those zones, how trades move between them, and how many crews each trade receives. These decisions are coupled: more zones can increase parallelism but also create more handoffs; more crews can reduce task durations but increase congestion and cost; and a different zone order can reduce idle time for one trade while increasing it for another.

In practice, these decisions are often made through planner experience, manual iteration, and intuition. While this expertise is valuable, it can make schedule quality hard to evaluate systematically. A schedule may look reasonable in a Gantt chart while still producing hidden inefficiencies such as crew waiting time, uneven workloads, disconnected work areas, or unnecessary resource use. These issues are especially important in MEP (mechanical, electrical and plumbing systems of a building) rough-in and interior construction, where many trades repeatedly work through the same rooms and corridors.

This project studies schedule optimization for the MEP rough-in and interiors phase of a building project. The building is represented as a set of BIM-derived rooms, each with

associated work quantities. A takt zone is a group of rooms planned and executed together. The scheduler uses room geometry, room quantities, trade productivity rates, and crew assumptions to generate a zone-based construction schedule, as shown in Fig. 1.



(a) Building Information Model



(b) Construction Schedule

Fig. 1: Building information model and corresponding construction schedule representation.

The central question is whether the schedule representation itself should be optimized: which rooms should be grouped into zones, in what order those zones should be executed, and how many crews each trade should receive. The proposed

approach treats this as a combined macro–micro optimization problem. At the macro level, the optimizer searches over room-to-zone grouping and zone execution order. At the micro level, it searches over crew counts. Each candidate plan is evaluated by a schedule simulator, and the resulting objectives are final duration, total idle time, spatial connectedness, and crew resource usage. The goal is to generate a Pareto set of feasible takt plans rather than a single manually tuned schedule.

## II. RELATED WORK

Construction scheduling is commonly treated as a resource- and dependency-constrained planning problem. Methods such as critical path scheduling and Gantt charts are useful for showing task order, but they are less effective at representing how crews physically move through repeated spaces such as rooms, floors, or zones [1]. Location-based scheduling and takt planning address this by modeling crew flow through space and emphasizing stable handoffs, balanced work packages, and predictable production rhythm [2]–[5].

Prior work shows that zone layout, workload balance, crew deployment, and schedule buffers strongly affect schedule performance [6]–[8]. However, many approaches assume that the basic schedule structure is already fixed. The optimizer may tune crew sizes, buffers, or activity timing, but the division of the building into zones and work packages is usually treated as an input.

This project instead treats BIM-derived takt planning as a representation-search problem. The idea is that the structure of the schedule itself can be optimized, similar to how shape grammar methods search over design structures [9] and Halide auto-scheduling searches over execution schedules such as tiling, ordering, and parallelization [10]. Analogously, this work searches over the construction schedule representation rather than only tuning a fixed plan.

Each candidate schedule is represented by two vectors: a room-to-zone vector and a crew-count vector. The room-to-zone vector defines which rooms are grouped together and the order in which those groups are executed. The crew-count vector defines how many crews are assigned to each trade. A schedule evaluator converts each candidate into task start times and durations, and the objective layer evaluates final duration, idle time, spatial connectedness, and crew resource cost.

The contribution is twofold: first, the optimizer searches over the macro structure of the schedule itself, including room grouping and zone order; second, it frames construction planning as a multiobjective optimization problem that produces a Pareto frontier of alternatives. This makes the tradeoffs explicit: faster schedules may require more crews, more handoffs, or less spatially compact work areas.

## III. METHODOLOGY

This project formulates construction schedule planning as a multiobjective optimization problem over schedule structure and resource allocation. The methodology has four main

components: a candidate schedule representation, a deterministic scheduler, a multiobjective performance metric, and an evolutionary search algorithm.

### A. Schedule Candidate Representation

Each candidate schedule is represented as  $x_i = (z_i, c_i)$  where  $z_i$  is the macro schedule representation and  $c_i$  is the micro resource representation.

The vector  $z_i = [z_{i,1}, z_{i,2}, \dots, z_{i,n}]$  assigns each room ( $r$ ) to a takt zone. Rooms with the same value of  $(z_{i,r})$  belong to the same zone. The value of  $(z_{i,r})$  also defines the execution order of the zone. For example, rooms assigned value 1 are executed before rooms assigned value 2.

The number of zones is not fixed. Instead, it is implied by the room-to-zone vector:  $p_i = |\text{unique}(z_i)|$

This allows the optimizer to compare schedules with different levels of spatial decomposition. A plan with more zones may allow more parallelism but may also increase handoffs and coordination burden. A plan with fewer zones may reduce fragmentation but may increase task durations and crew waiting.

The crew-count vector is:  $c_i = [c_{i,1}, c_{i,2}, \dots, c_{i,m}]$ , where  $c_{i,k}$  is the number of crews assigned to trade  $k$ . Crew counts are bounded by  $1 \leq c_{i,k} \leq C_k^{\max}$ .

This representation captures both the macro decision of how the building is decomposed into zones and the micro decision of how resources are assigned to trades.

### B. Scheduler

Each candidate is built into a schedule. The scheduler receives the candidate plan ( $x_i$ ), BIM-derived room quantities ( $\mathbf{q}_i$ ), and trade productivity rates ( $\rho_i$ ). It returns the start-time ( $\mathbf{s}_i$ ) and duration matrices ( $\mathbf{d}_i$ ):  $\mathcal{S}(x_i; \mathbf{q}, \rho) \rightarrow (\mathbf{s}_i, \mathbf{d}_i)$ .

For each candidate  $i$ , zone  $j$ , and trade  $k$ , the zone-trade quantity is given by Eq. 1.

$$Q_{i,j,k} = \sum_{r=1}^n \mathbf{1}[z_{i,r} = j] q_{r,k} \quad (1)$$

The corresponding task duration is given by Eq. 2.

$$d_{i,j,k} = \frac{Q_{i,j,k}}{\rho_k c_{i,k}} \quad (2)$$

Then tasks are scheduled using two precedence assumptions. First, each zone follows a fixed trade sequence. Second, each trade flows from one zone to the next. The start time is therefore computed recursively as shown in Eq. 3, with boundary values equal to zero.

$$s_{i,j,k} = \max(s_{i,j,k-1} + d_{i,j,k-1}, s_{i,j-1,k} + d_{i,j-1,k}) \quad (3)$$

### C. Objective Functions

Each candidate is evaluated using four objectives, as shown in Equation 4a. Here,  $T_i$  is the final project duration,  $I_i$  is the total idle time,  $P_{\text{space},i}$  measures the spatial connectivity of the takt zones, and  $P_{\text{resource},i}$  measures crew cost. In Equation 4d,  $\kappa(Z_{i,j})$  is the number of connected components in zone  $j$ .

These four objectives were chosen because they represent the main planning tradeoffs: schedule speed, workflow stability, spatial feasibility, and resource burden.

$$J(x_i) = [T_i, I_i, P_{space,i}, P_{resource,i}] \quad (4a)$$

$$T_i = \max_{j,k} f_{i,j,k} \quad (4b)$$

$$I_i = \sum_{k=1}^m \sum_{j=2}^{p_i} (s_{i,j,k} - f_{i,j-1,k}) \quad (4c)$$

$$P_{space,i}^{conn} = \sum_{j=1}^{p_i} (\kappa(Z_{i,j}) - 1) \quad (4d)$$

$$P_{resource,i} = \sum_{k=1}^m c_{i,k} \quad (4e)$$

#### D. Initial Candidate Generation

The initial population is generated to cover a diverse range of schedule structures. Because the number of zones ( $p$ ) is not fixed, candidates are stratified across 1 to the total number of rooms. For each selected value of  $p$ , an initial room grouping is generated using a workload-aware spatial heuristic. First, each room is assigned an approximate workload as in Eq. 5 and an overall target workload is defined through Eq. 6.

$$w_r = \sum_{k=1}^m \frac{q_{r,k}}{\rho_k} \quad (5)$$

$$W_{target} = \frac{\sum_{r=1}^n w_r}{p} \quad (6)$$

Initial zones are then generated using a capacitated region-growing heuristic on the room adjacency graph. Seed rooms are selected across the floor, and zones grow by adding adjacent rooms that best preserve workload balance and spatial compactness. The incremental cost of assigning room ( $r$ ) to zone ( $j$ ) is defined in Eq.

$$\begin{aligned} \Delta(r, j) = & \alpha \left[ (W_j + w_r - W_{target})^2 - (W_j - W_{target})^2 \right] \\ & + \gamma \|\mathbf{x}_r - \bar{\mathbf{x}}_j\|^2 \\ & - \eta A(r, j) \end{aligned} \quad (7)$$

where  $\gamma \|\mathbf{x}_r - \bar{\mathbf{x}}_j\|^2$  penalizes assigning room  $r$  to zone  $j$  when the room centroid  $\mathbf{x}_r$  is far from the current zone centroid  $\bar{\mathbf{x}}_j$  and  $A(r, j)$  measures how many rooms already assigned to zone  $j$  are adjacent to candidate room  $r$ , as shown in Eq. 8.

$$A(r, j) = |\{r' \in Z_j : (r, r') \in E\}| \quad (8)$$

For each generated room-to-zone assignment  $z$ , an ordered set of candidate zone sequences is initialized using spatial sweep directions. First, room centroid coordinates  $\mathbf{x}_r = (x_r, y_r)$  are read from the room boundary data for Level 1.

Rooms are grouped by their current zone label, and each zone centroid is computed as the mean location of the rooms assigned to that zone:

$$\bar{\mathbf{x}}_j = \frac{1}{|Z_j|} \sum_{r \in Z_j} \mathbf{x}_r.$$

To generate multiple possible execution orders, the algorithm defines sweep vectors every  $45^\circ$  around the unit circle:

$$\mathbf{v}_\theta = [\cos(\theta), \sin(\theta)], \quad \theta \in \{0^\circ, 45^\circ, \dots, 315^\circ\}.$$

For each sweep direction, every zone is assigned a scalar projection score:

$$a_j^{(\theta)} = \bar{\mathbf{x}}_j^\top \mathbf{v}_\theta.$$

Zones are then sorted by this projection value, producing an execution order along that spatial direction. This produces multiple spatially reasonable initial orderings of the same zone grouping, which can then be evaluated and improved by the optimizer.

Next, the crew initializer creates three baseline crew-count. For each trade  $k$ , there exists a maximum available crew count. The initializer then creates one maximum-resource candidate, one minimum-resource candidate, and one midpoint candidate.

#### E. Multiobjective Evolutionary Search

The optimization is performed using an NSGA-II-style evolutionary algorithm. NSGA-II is used because the project does not seek a single weighted optimum. Instead, it seeks a Pareto set of schedules that expose the tradeoffs between duration, idle time, spatial quality, and resource usage.

For each generation, all candidates are evaluated by the scheduler and assigned an objective vector ( $J(x_i)$ ). Candidates are then sorted using Pareto dominance. A candidate ( $x_a$ ) dominates candidate ( $x_b$ ) if:  $f_l(x_a) \leq f_l(x_b) \quad \forall l$  and  $\exists l : f_l(x_a) < f_l(x_b)$ .

Within each Pareto front, crowding distance is used to preserve diversity. All candidates in the same front have the same Pareto rank, so crowding distance acts as a tie-breaker: candidates that lie in less crowded regions of the front are preferred over candidates that are very close to many others. For each front, the crowding distance of every candidate is first initialized to zero. Then, for each objective, the candidates are sorted by their objective value. The two boundary candidates, corresponding to the minimum and maximum value of that objective, are assigned infinite crowding distance so that extreme tradeoff solutions are preserved.

For every non-boundary candidate, the crowding distance is increased by the normalized gap between its two neighboring candidates in the sorted list as in Eq. 9.

$$CD(x_i) \leftarrow CD(x_i) + \frac{f_k(x_{i+1}) - f_k(x_{i-1})}{f_k^{\max} - f_k^{\min}} \quad (9)$$

Here,  $f_k(x_i)$  is the value of objective  $k$  for candidate  $x_i$ , and  $x_{i-1}$  and  $x_{i+1}$  are the neighboring candidates after sorting the front by objective  $k$ . This process is repeated for all objectives: final duration, idle time, space penalty, and resource penalty.

### F. Crossover

Tournament selection is used to choose two parents ( $x^A$  and  $x^B$ ) by randomly sampling two candidates at a time and selecting the candidate with the better NSGA-II ordering, prioritizing lower Pareto rank and then higher crowding distance. Crossover uses objective-specific strengths to construct a child zoning plan. A parent is assigned for BLOCK, FILL, and ORDER based on different objective strengths. BLOCK signifies the parent from which the child inherits a fraction  $\rho$  of whole zone blocks. FILL defines the parent used to assign the remaining unassigned rooms after the inherited blocks have been copied. ORDER defines the parent whose zone sequence is used to relabel the child zones into an execution order. For a child zone  $Z_j$ , its ordering score is in Eq. 10. Zones are then relabeled in ascending order of  $s_j$ , producing contiguous labels  $1, 2, \dots, p$ .

$$s_j = \frac{1}{|Z_j|} \sum_{r \in Z_j} z_r^{\text{dur}} \quad (10)$$

Crew crossover is performed independently for each trade:

$$c_k^{\text{child}} = \begin{cases} c_k^A, & \text{with probability 0.5} \\ c_k^B, & \text{with probability 0.5} \end{cases}$$

The resulting crew count is clamped to the feasible range:

$$1 \leq c_k^{\text{child}} \leq C_k^{\text{max}}$$

### G. Mutation

After crossover, one mutation operator is applied to the child. The mutation family is selected probabilistically based on the candidate's normalized objective weaknesses. For objective  $m$ , the normalized score is in Eq. 11, where  $f_m^{\text{min}}$  and  $f_m^{\text{max}}$  are computed over the current population.

$$\hat{f}_m(x) = \frac{f_m(x) - f_m^{\text{min}}}{f_m^{\text{max}} - f_m^{\text{min}}} \quad (11)$$

The probability of selecting mutation family  $m$  is given in Eq. 12.

$$P(m) = \frac{\hat{f}_m(x)}{\sum_q \hat{f}_q(x)} \quad (12)$$

If all normalized scores are zero, the mutation family is selected uniformly at random.

The mutation families are mapped to schedule deficiencies as follows:

- $T(x) \rightarrow$  split overloaded zone or add crew
- $I(x) \rightarrow$  zone order swap, insert, or reversal
- $P_{\text{conn}}(x) \rightarrow$  boundary-room move or swap
- $C(x) \rightarrow$  merge underloaded zones or remove crew

After crossover and mutation, each child is checked for feasibility. A candidate is valid only if every room appears exactly once, no zone is empty, zone labels are contiguous, and crew counts remain within bounds.

### H. Stopping Criteria

The optimization loop is terminated using both a hard computational budget and a convergence-based stopping criterion. The hard budget prevents excessive computation: the algorithm stops if either the maximum number of generations or the maximum number of candidate evaluations is reached.

In addition, convergence is monitored using the change in hypervolume over a fixed window of  $K$  generations. Let  $F_t$  denote the set of non-dominated candidates at generation  $t$ , and let  $HV_t$  denote the hypervolume of this set. The optimization is stopped if the relative improvement in hypervolume over the last  $K$  generations is smaller than a threshold  $\epsilon$ :

$$\frac{|HV_t - HV_{t-K}|}{|HV_{t-K}|} < \epsilon. \quad (13)$$

This criterion indicates that the Pareto front is no longer improving meaningfully across the monitored objectives.

Before computing hypervolume, each objective is normalized using the minimum and maximum values observed in the current population. Using the normalized objective vector  $\hat{\mathbf{f}}(x)$ , the hypervolume is approximated as

$$HV_t = \sum_{x \in F_t} \prod_{m=1}^M (1 - \hat{f}_m(x)). \quad (14)$$

In this implementation,  $M = 4$ , corresponding to final duration, idle time, space penalty, and resource penalty. Since all objectives are minimized, the term  $1 - \hat{f}_m(x)$  is larger when candidate  $x$  is closer to the best observed value for objective  $m$ . Therefore, a larger hypervolume indicates that the current non-dominated set contains candidates that are closer to the ideal objective values and cover a larger favorable region of the objective space.

## IV. RESULTS AND DISCUSSION

This section evaluates which evolutionary design choices produce the strongest schedule candidates. The ablation study compares mutation and crossover variants, while the sensitivity study tests selected inheritance and population parameters. Each row reports the mean objective values of the best candidate found across 10 independent runs. These experiments are used to select the optimizer configuration; the final baseline comparison is reported separately. Across all tables, lower values are preferred for duration, idle time, connectedness penalty, and crew cost.

### A. Ablation and Sensitivity Results

Table I shows that each mutation family contributes to a different aspect of schedule quality. Removing crew mutation produces the largest increase in duration, from 229.87 to 292.07, which suggests that crew-count changes are important for reducing critical-path delays. Removing order mutation increases idle time from 91.76 to 115.08, showing that changes in zone sequencing help smooth the movement of trades between zones. Removing zone mutation produces the largest idle-time penalty, increasing idle time to 137.42. This suggests

that changing the room-to-zone grouping is important for reducing waiting caused by imbalanced or poorly structured zones. Overall, the full mutation model gives the best balance across duration, idle time, connectedness, and crew cost.

TABLE I: Mutation ablation results

| Variant           | Duration      | Idle         | Conn. Penalty | Crew Cost    |
|-------------------|---------------|--------------|---------------|--------------|
| <b>Full model</b> | <b>229.87</b> | <b>91.76</b> | <b>0.00</b>   | <b>16.70</b> |
| No crew mutation  | 292.07        | 99.89        | 0.20          | 17.20        |
| No order mutation | 237.22        | 115.08       | 0.00          | 15.40        |
| No zone mutation  | 248.96        | 137.42       | 0.00          | 14.20        |

Table II shows that crossover design has a strong effect on the quality of the generated schedules. Random-block crossover achieves a relatively low duration of 176.42, but it increases idle time and produces a connectedness penalty of 0.40. This means that disruptive recombination can find fast schedules, but it can also damage the spatial coherence of the work zones.

The best overall tradeoff is produced by the crossover variant that inherits connected zone blocks, fills the remaining rooms using duration performance, and orders the resulting zones using idle-time performance. This strategy reduces duration to 162.80 and idle time to 88.53 while maintaining a connectedness penalty of 0.00. Although the “block duration, fill idle, order connectedness” variant achieves the lowest duration, 160.86, it produces much higher idle time, 149.69. Therefore, the selected crossover strategy is not the absolute fastest schedule, but the most balanced operator configuration across schedule speed, crew flow, and spatial feasibility.

TABLE II: Crossover ablation results

| Variant                                       | Duration      | Idle         | Conn.       | Crew         |
|-----------------------------------------------|---------------|--------------|-------------|--------------|
| Current                                       | 229.87        | 91.76        | 0.00        | 16.70        |
| Conn. first                                   | 213.25        | 145.50       | 0.20        | 16.10        |
| Duration first                                | 199.66        | 134.94       | 0.00        | 14.50        |
| Random blocks                                 | 176.42        | 121.27       | 0.40        | 17.40        |
| <b>Block conn., fill duration, order idle</b> | <b>162.80</b> | <b>88.53</b> | <b>0.00</b> | <b>16.43</b> |
| Block duration, fill idle, order conn.        | 160.86        | 149.69       | 0.00        | 16.57        |
| Block idle, fill duration, order conn.        | 201.40        | 91.43        | 0.14        | 19.71        |

Table III shows that the optimizer is sensitive to inheritance and population settings. An inheritance ratio of 0.40 produces the lowest idle time, suggesting that moderate inheritance preserves useful parent structure while still allowing enough recombination to improve flow. In contrast, an inheritance ratio of 0.80 lowers crew cost but worsens both duration and idle time, indicating that excessive inheritance may reduce exploration and preserve inefficient schedule structure.

The population-size results show a similar tradeoff. A population size of 10 performs worse in duration and connectedness, likely because the search has limited diversity. A population size of 20 lowers crew cost but increases idle time, suggesting that larger populations can discover lower-resource schedules but do not automatically improve flow without sufficient generations or stronger selection pressure.

TABLE III: Parameter sensitivity results.

| Variant                | Duration | Idle   | Conn. Penalty | Crew Cost |
|------------------------|----------|--------|---------------|-----------|
| Inheritance ratio 0.40 | 235.86   | 56.97  | 0.00          | 16.70     |
| Inheritance ratio 0.80 | 253.08   | 167.85 | 0.20          | 13.50     |
| Population size 10     | 262.50   | 127.86 | 0.10          | 15.30     |
| Population size 20     | 241.88   | 201.61 | 0.00          | 12.70     |

Overall, these ablation and sensitivity results support the central claim that schedule quality depends on both resource allocation and representation design. Crew mutation primarily affects duration, order mutation affects trade waiting time, and zone mutation affects the spatial structure of the schedule. Similarly, the strongest crossover strategy is the one that combines spatial validity, duration performance, and idle-time reduction rather than optimizing only one objective. The selected operator configuration is therefore used for the final optimized schedule comparison in the next section.

### B. Optimized Schedule Performance

After the ablation study, the final optimizer was run using the selected crossover strategy: connectedness-based block inheritance, duration-based filling, and idle-time-based zone ordering. The baseline uses a manually defined two-room zig-zag zoning strategy with fixed crew counts. The selected optimized schedule achieved a duration of 202.70, idle time of 2.30, connectedness penalty of 0, and crew cost of 16. Compared with the baseline schedule, developed manually by an average-skilled construction manager, the optimized solution eliminates disconnected zones and substantially reduces idle time, while using additional crew resources to improve workflow continuity.

TABLE IV: Baseline and optimized schedule comparison.

| Schedule         | Duration      | Idle          | Conn. Penalty | Crew Cost    |
|------------------|---------------|---------------|---------------|--------------|
| Baseline         | 258.82        | 464.92        | 2.00          | 10.00        |
| <b>Optimized</b> | <b>221.40</b> | <b>111.25</b> | <b>0.00</b>   | <b>11.00</b> |

Figure 2 shows the optimized zone layout. Each room is labeled with its assigned zone, and the color gradient represents the execution order. The optimized layout maintains spatially coherent zones while preserving a feasible sequence for takt execution.

Figure 3 shows the optimized takt plan. Each colored bar represents active work by a trade, while semi-transparent bars indicate idle time for that trade. The optimized schedule shows substantially reduced idle time compared with the baseline takt plan in Figure 4.

Finally, Figure 5 shows the tradeoff between duration and crew cost across the generated candidate population. Marker size represents idle time, and the red star marks the final selected solution. This plot highlights the multi-objective nature of the problem: lower duration generally requires additional crew resources, while the selected solution balances duration, idle time, connectedness, and crew cost.

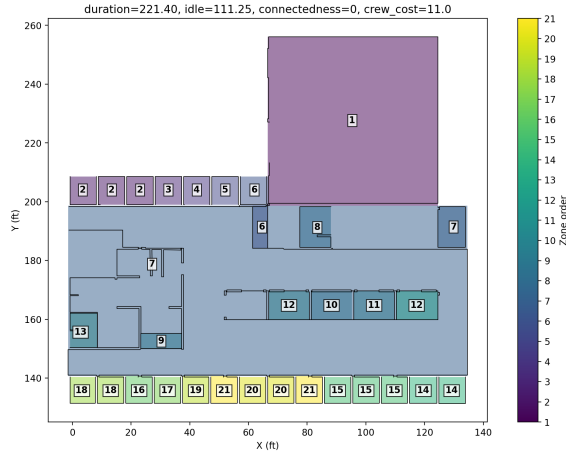


Fig. 2: Optimized zoning plan. Rooms are labeled by zone ID and colored by execution order.

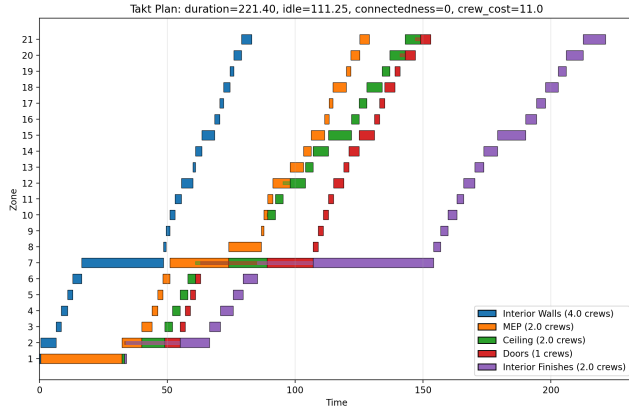


Fig. 3: Optimized takt plan. Bars are colored by trade, with crew counts shown in the legend.

## V. CONCLUSION AND FUTURE WORK

Overall, the ablation results show that the optimizer performs best when spatial validity and schedule performance are handled together rather than independently. Connected zone inheritance preserves feasible work areas, duration-based filling improves schedule efficiency, and idle-based ordering improves crew flow. The final crossover strategy therefore provides the most balanced performance across duration, idle time, connectedness, and crew cost, making it the most suitable configuration for the proposed takt planning optimizer. Future work should extend the optimizer to larger BIM models, incorporate uncertainty in productivity rates and crew availability, and validate the generated Pareto alternatives against planner feedback and real project constraints.

## REFERENCES

- [1] M. A. Ammar, "Lob and cpm integrated method for scheduling repetitive projects," *Journal of construction engineering and management*, vol. 139, no. 1, pp. 44–50, 2013.
- [2] R. Kenley and O. Seppänen, *Location-Based Management for Construction: Planning, Scheduling and Control*. London and New York: Spon Press, 2010.

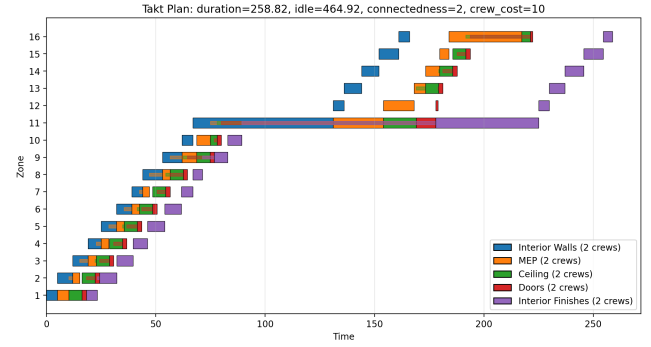


Fig. 4: Baseline takt plan using two-room zig-zag zoning and two crews per trade.

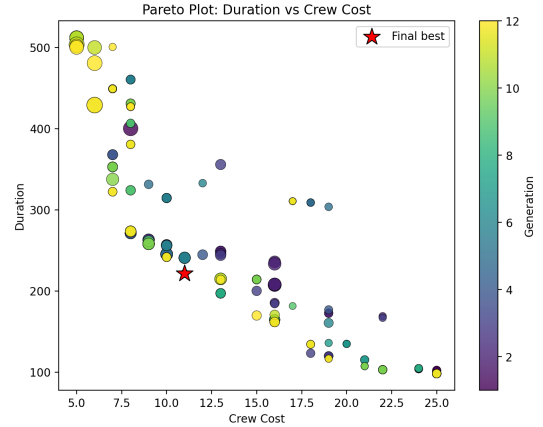


Fig. 5: Pareto plot of duration versus crew cost. Marker size is proportional to idle time, and the red star marks the selected solution.

- [3] O. Seppänen, G. Ballard, and S. Pesonen, "The combination of last planner system and location-based management system," in *Proceedings of the 18th Annual Conference of the International Group for Lean Construction (IGLC)*, (Haifa, Israel), 2010.
- [4] A. G. Frandson, *Takt Time Planning as a Work Structuring Method to Improve Construction Work Flow*. Phd dissertation, University of California, Berkeley, 2019.
- [5] M. E. Vatne and F. Drevland, "Practical benefits of using takt time planning: A case study," in *Proceedings of the 24th Annual Conference of the International Group for Lean Construction (IGLC)*, (Boston, USA), 2016.
- [6] K. El-Rayes and O. Moselhi, "Minimizing duration and crew work interruptions of repetitive construction projects," *Automation in Construction*, vol. 10, no. 4, pp. 433–446, 2001.
- [7] I. Bakry, O. Moselhi, and T. Zayed, "Optimizing the scheduling of crew deployments in repetitive construction projects," *Engineering, Construction and Architectural Management*, vol. 27, no. 6, pp. 1615–1636, 2020.
- [8] A. Jabbari, *Space Planning for Takt Production*. Phd thesis, University of California, Berkeley, 2020.
- [9] M. Kodnongbua, Z. J. Zhang, N. Sharp, and A. Schulz, "Design for descent: What makes a shape grammar easy to optimize?," in *SIGGRAPH Asia 2025 Conference Papers*, 2025.
- [10] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand, and J. Ragan-Kelley, "Learning to optimize Halide with tree search and random programs," *ACM Transactions on Graphics*, vol. 38, no. 4, 2019.